



Big Data Storage and Processing

Topic: Traffic Accident Analysis

Phan Công Anh
Lê Quang Hiếu
Nguyễn Việt Dũng
Bùi Anh Nhật
Nhữ Minh Hà
Vũ Nhật Minh

20210078
20210335
20214883
20210657
20212784
2021????

Instructor: Prof. Tran Viet Trung

Dec, 2025

Contents

1	Problem Statement	3
1.1	Problem	3
1.2	Motivation	3
2	Dataset	3
3	Technologies Used and System Architecture	4
4	System Component Details	5
4.1	Kafka	5
4.1.1	Kafka	5
4.1.2	Kafka Producer	7
4.1.3	Kafka Consumer	7
4.2	HDFS	8
4.3	Spark	10
4.3.1	Spark Streaming	10
4.4	Cassandra	11
4.5	Visualization	13
5	Conclusion	14
6	Problem and Solutions	14

1 Problem Statement

1.1 Problem

This project focuses on building a system for aggregating and analyzing traffic accident data using big data storage and processing techniques. The goal is to create an efficient, scalable system capable of automating the collection, storage, and analysis of traffic accident data from various sources across the United States.

1.2 Motivation

With the rapid increase in traffic data generated across the U.S., managing and analyzing this information requires a powerful system capable of processing large datasets. Analyzing traffic accident data can provide critical insights into accident patterns, contributing to better decision-making for traffic management and road safety improvements. This system aims to deliver accurate and timely data analysis, helping users access valuable insights quickly and efficiently.

2 Dataset

The US Accidents Dataset from Kaggle includes over 2.8 million records of traffic accidents occurring across the United States from February 2016 to June 2021. Each record contains detailed information about an accident incident, with the following fields:

- Severity: Accident severity on a scale from 1 (least severe) to 4 (most severe).
- Start Time: Date and time when the accident occurred.
- End Time: Date and time when the accident was resolved.
- Start Location (Latitude & Longitude): Geographic coordinates where the accident started.
- End Location (Latitude & Longitude): Geographic coordinates where the accident ended (if available).
- Description: A brief description of the accident.
- Street, City, State, Country: Specific location details.
- Weather Conditions: Weather during the time of the accident.
- Visibility, Wind Speed, Temperature: Environmental conditions.
- Traffic Signal: Whether there were traffic signals at the accident site.
- Distance (mi): Length of the affected road segment.
- Source: The source of the reported accident data.

The dataset provides a comprehensive view of traffic incidents across the U.S., enabling detailed analysis for various applications such as accident prediction, route optimization, and safety assessments.

3 Technologies Used and System Architecture

1. Kubernetes (K8s):

- Function: Orchestrates containers, manages deployment, scaling, and operation of system components.
- Benefits: Provides high availability, scalability, and easier management of microservices.

2. Kafka:

- Function: Processes streaming data, ensuring continuous and synchronized information delivery.
- Benefits: Supports event-driven architecture and real-time data processing.

3. Spark Streaming:

- Function: Processes and transforms streaming data from Kafka.
- Benefits: Enables real-time data analytics and continuous data processing.

4. Spark:

- Function: Processes, transforms, and analyzes batch data from HDFS.
- Benefits: Suitable for large-scale processing and supports complex tasks.

5. Cassandra:

- Function: Stores processed data from Spark and Spark Streaming.
- Benefits: Provides high scalability and performance, suitable for distributed data storage.

6. Trino:

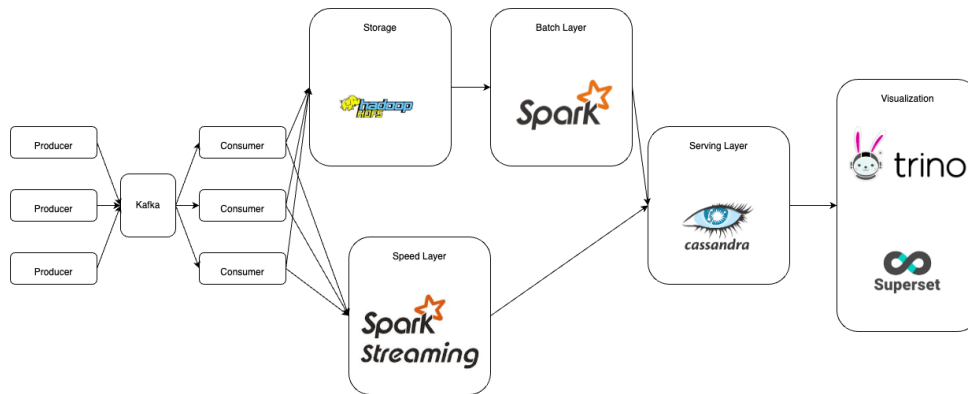
- Function: Convert NoSQL to SQL.
- Benefits: Fast queries, supports multiple data sources, and scales horizontally.

7. Superset:

- Function: Visualizes real-time and historical data through interactive dashboards.
- Benefits: Real-time monitoring and alerts; Customizable dashboards.

8. Lambda Architecture:

- Description: The system is built following the Lambda Architecture, which is stream processing.
- Benefits: Leverages Spark Streaming for real-time processing.



Hình 1: System Design

The system follows the Kappa Architecture, where data is processed through stream processing with Spark Streaming. Processed data is stored in Cassandra and later retrieved for visualization. An overall system architecture diagram is illustrated above.

4 System Component Details

4.1 Kafka

4.1.1 Kafka

Our Kafka cluster contains 4 nodes: 3 brokers and 1 zookeeper. Zookeeper configuration:

```

# Zookeeper StatefulSet
apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: zookeeper
  namespace: bigdata-pipeline
spec:
  serviceName: zookeeper
  replicas: 1
  selector:
    matchLabels:
      app: zookeeper
  template:
    metadata:
      labels:
        app: zookeeper
    spec:
      containers:
        - name: zookeeper
          image: confluentinc/cp-zookeeper:5.1.0
          ports:
            - containerPort: 2181
              name: client
---
# Zookeeper Service
apiVersion: v1

```

```

kind: Service
metadata:
  name: zookeeper
  namespace: bigdata-pipeline
spec:
  type: ClusterIP
  selector:
    app: zookeeper
  ports:
    - port: 2181
      targetPort: 2181
      name: client

```

Broker configuration:

```

apiVersion: apps/v1
kind: Deployment
metadata:
  name: kafka-1
  namespace: bigdata-pipeline
spec:
  replicas: 1
  selector:
    matchLabels:
      app: kafka-1
  template:
    metadata:
      labels:
        app: kafka-1
    spec:
      containers:
        - name: kafka
          image: confluentinc/cp-kafka:5.1.0
          ports:
            - containerPort: 9092
          env:
            - name: KAFKA_BROKER_ID
              value: "1"
            - name: KAFKA_ZOOKEEPER_CONNECT
              value: "zookeeper:2181"
            - name: KAFKA_ADVERTISED_LISTENERS
              value: "PLAINTEXT://kafka-1:9092"
            - name: KAFKA_LISTENER_SECURITY_PROTOCOL_MAP
              value: "PLAINTEXT:PLAINTEXT"
            - name: KAFKA_INTER_BROKER_LISTENER_NAME
              value: "PLAINTEXT"
            - name: KAFKA_OFFSETS_TOPIC_REPLICATION_FACTOR
              value: "2"

```

The other 2 brokers are configured in the same way. In this project, the group uses one topic with three partitions and a replication factor of two.

```
kubectl exec -n bigdata-pipeline <POD_NAME> -- /usr/bin/kafka-topics \
```

```
--create \
--zookeeper zookeeper:2181 \
--partitions 3 \
--replication-factor 2 \
--topic trafficaccident
```

Code used to describe topic:

```
bootstrap_servers = 'kafka-1:9092,kafka-2:9093,kafka-3:9094'
admin_client = KafkaAdminClient(bootstrap_servers=bootstrap_servers)

kafka_topic = 'TrafficAccident'
topic = NewTopic(name=kafka_topic, num_partitions=3, replication_factor=2)
try:
    admin_client.create_topics([topic])
    print(f"Topic '{kafka_topic}' created successfully")
except Exception as e:
    print(f"Topic creation error (may already exist): {e}")

try:
    topic_details = admin_client.describe_topics([topic_name])
    print(f"Details for topic '{topic_name}':")
    for detail in topic_details:
        print(detail)
except Exception as e:
    print(f"Loi '{topic_name}': {e}")
```

Status output:

```
{'error_code': 0, 'topic': 'trafficaccident', 'is_internal': False,
 'partitions': [{'error_code': 0, 'partition': 0, 'leader': 3,
                  'replicas': [3, 1], 'isr': [3, 1], 'offline_replicas': []},
                 {'error_code': 0, 'partition': 2, 'leader': 2,
                  'replicas': [2, 3], 'isr': [2, 3], 'offline_replicas': []},
                 {'error_code': 0, 'partition': 1, 'leader': 1,
                  'replicas': [1, 2], 'isr': [1, 2], 'offline_replicas': []}]
}
```

Status output when a Kafka broker is down:

```
Details for topic 'trafficaccident':
{'error_code': 0, 'topic': 'trafficaccident', 'is_internal': False,
 'partitions': [{'error_code': 0, 'partition': 0, 'leader': 1,
                  'replicas': [3, 1], 'isr': [1], 'offline_replicas': []},
                 {'error_code': 0, 'partition': 2, 'leader': 2,
                  'replicas': [2, 3], 'isr': [2], 'offline_replicas': [3]},
                 {'error_code': 0, 'partition': 1, 'leader': 1,
                  'replicas': [1, 2], 'isr': [1, 2]}]
}
```

4.1.2 Kafka Producer

Our group simulates a data stream continuously sent into the system through a producer. First, the dataset is downloaded. The producer process reads the entire dataset and sim-

ulates the data stream by publishing an article to the broker in JSON format every 0.5 seconds.

```
# Kafka broker addresses for Kubernetes cluster
# Using service names within the bigdata-pipeline namespace
# Format: kafka-1:9092,kafka-2:9093,kafka-3:9094
bootstrap_servers = 'kafka-1:9092,kafka-2:9093,kafka-3:9094'
admin_client = KafkaAdminClient(bootstrap_servers=bootstrap_servers)

# Kafka topic to publish to
kafka_topic = 'TrafficAccident'
topic = NewTopic(name=kafka_topic, num_partitions=3, replication_factor=2)

try:
    admin_client.create_topics([topic])
    print(f"Topic '{kafka_topic}' created successfully")
except Exception as e:
    print(f"Topic creation error (may already exist): {e}")

# Read JSON data from the file
data_source_path = os.path.join(os.path.dirname(__file__), '2022-09-06.csv')
csv_data = pandas.read_csv(data_source_path)
csv_data = csv_data[['ID', 'Severity', 'Start_Time', 'Weather_Condition']]
csv_data = csv_data.to_dict('records')

# Kafka producer configuration
producer = KafkaProducer(bootstrap_servers=bootstrap_servers)

# Function to publish an article to Kafka topic
def send_a_row(row):
    key = str(row["ID"]).encode('utf-8') # Encode the key to bytes
    value = json.dumps(row).encode('utf-8')
    producer.send(kafka_topic, key=key, value=value)
    producer.flush()
    print('sent one row')

# Publish an article every 0.5 seconds
for row in csv_data:
    send_a_row(row)
    time.sleep(0.5)

# Close the Kafka producer
producer.close()
```

4.1.3 Kafka Consumer

The consumer is located in the kafka-consumer pod within the Kubernetes cluster. It subscribes to the TrafficAccident topic across multiple Kafka brokers (kafka-1, kafka-2, kafka-3), retrieves the JSON-encoded data, and decodes it into a dictionary format. The

consumer implements a batching mechanism that collects up to 100 rows (or waits for 30 seconds) before using a SparkSession to convert the accumulated data into a DataFrame. Once processed, it converts the batch into CSV format and saves it to HDFS.

```
# Create SparkSession once and reuse it (more efficient)
spark = SparkSession.builder \
    .appName("consumer") \
    .config("spark.driver.memory", "512m") \
    .config("spark.driver.maxResultSize", "256m") \
    .config("spark.executor.memory", "512m") \
    .config("spark.sql.shuffle.partitions", "2") \
    .getOrCreate()

# Batch configuration
BATCH_SIZE = 100 # Number of messages to batch before writing
BATCH_TIMEOUT = 30 # Maximum seconds to wait before flushing batch
MAX_RETRIES = 3 # Maximum retry attempts for HDFS writes
RETRY_DELAY = 5 # Seconds to wait between retries

def write_batch_to_hdfs(batch_data, retry_count=0):
    """
    Write a batch of data to HDFS with retry logic
    """
    if not batch_data:
        return True

    try:
        # Convert batch to dataframe
        df = spark.createDataFrame(batch_data)

        # Create path with timestamp (one file per batch, not per message)
        now = datetime.datetime.now()
        path = 'hdfs://namenode:9000/user/pdt/raw_rows/batch_{}_{}_{}_{}_{}_{}_{}_{}.csv'.format(
            now.year, now.month, now.day, now.hour, now.minute, now.second, now.
            microsecond
        )

        # Write to HDFS with error handling
        df.write \
            .option("header", "true") \
            .option("delimiter", "|||") \
            .csv(path)

        print(f'[kafka consumer saved batch of {len(batch_data)} rows]')
        return True

    except Exception as e:
        print(f'[ERROR] Failed to write batch to HDFS: {str(e)}')
        if retry_count < MAX_RETRIES:
            print(f'[RETRY] Retrying in {RETRY_DELAY} seconds... (attempt {
                retry_count + 1}/{MAX_RETRIES})')
```

```

        time.sleep(RETRY_DELAY)
        return write_batch_to_hdfs(batch_data, retry_count + 1)
    else:
        print(f'[ERROR] Max retries reached. Dropping batch of {len(
            batch_data)} rows.')
        return False

print('hello')

# Kafka consumer configuration
# Using all Kafka brokers in Kubernetes cluster for better reliability
consumer_config = {
    'bootstrap_servers': 'kafka-1:9092,kafka-2:9093,kafka-3:9094', # All
        brokers in Kubernetes cluster
    'group_id': 'my_consumer_group',
    'auto_offset_reset': 'latest',
    'enable_auto_commit': True,
    'auto_commit_interval_ms': 100,
}

print("configed")

# Create Kafka consumer instance
consumer = KafkaConsumer(**consumer_config)

print("created")

# Subscribe to a Kafka topic
kafka_topic = 'TrafficAccident' # Replace with your Kafka topic
consumer.subscribe([kafka_topic])

print("subscribed")

# Batch buffer
batch_buffer = []
last_batch_time = time.time()

try:
    while True:
        records = consumer.poll(1000) # timeout in milliseconds
        # Process received records
        for tp, consumer_records in records.items():
            for consumer_record in consumer_records:
                try:
                    # Convert from byte -> dict (json format)
                    decoded_message = consumer_record.value.decode('utf-8')
                    data_dict = json.loads(decoded_message)

                    # Add to batch buffer
                    batch_buffer.append(data_dict)
                    print(f'[kafka consumer received row] (batch size: {len(

```

```

        batch_buffer)))

    except Exception as e:
        print(f'[ERROR] Failed to process message: {str(e)}')
        continue

    # Check if we should flush the batch
    current_time = time.time()
    should_flush = (
        len(batch_buffer) >= BATCH_SIZE or
        (len(batch_buffer) > 0 and (current_time - last_batch_time) >=
         BATCH_TIMEOUT)
    )

    if should_flush and batch_buffer:
        success = write_batch_to_hdfs(batch_buffer)
        if success:
            batch_buffer = [] # Clear buffer on success
            last_batch_time = current_time
        # If failed, keep buffer for next retry (but don't block forever)

except KeyboardInterrupt:
    print('\n[INFO] Shutting down gracefully...')
    # Write remaining batch before exit
    if batch_buffer:
        print(f'[INFO] Flushing remaining {len(batch_buffer)} rows...')
        write_batch_to_hdfs(batch_buffer)
except Exception as e:
    print(f'[FATAL ERROR] Unexpected error: {str(e)}')
    # Try to save remaining batch
    if batch_buffer:
        print(f'[INFO] Attempting to save remaining {len(batch_buffer)} rows
            ...')
        write_batch_to_hdfs(batch_buffer)

```

4.2 HDFS

The group uses HDFS to store the information of articles received from the consumer and to save the processing results from Spark. A Hadoop cluster with four nodes is set up: one namenode and three datanodes. The replication factor is configured to be 3. Each piece of data stored in HDFS is divided into blocks, with each block having a size of 128 MB and three replications.

4.2.1 Namenode

```

apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: namenode
  namespace: bigdata-pipeline

```

```

spec:
  serviceName: namenode
  replicas: 1
  selector:
    matchLabels:
      app: namenode
  template:
    metadata:
      labels:
        app: namenode
    spec:
      containers:
        - name: namenode
          image: bde2020/hadoop-namenode:2.0.0-hadoop3.2.1-java8
          ports:
            - containerPort: 9870
              name: web
            - containerPort: 9000
              name: rpc
          env:
            - name: CLUSTER_NAME
              value: "test"
            - name: CORE_CONF_fs_defaultFS
              value: "hdfs://namenode:9000"
            - name: CORE_CONF_hadoop_http_staticuser_user
              value: "root"
            - name: HDFS_CONF_dfs_webhdfs_enabled
              value: "true"
            - name: HDFS_CONF_dfs_permissions_enabled
              value: "false"
            - name:
                HDFS_CONF_dfs_namenode_datanode_registration_ip___hostname___check

              value: "false"
      resources:
        requests:
          memory: "512Mi"
          cpu: "100m"
        limits:
          memory: "2Gi"
          cpu: "1000m"
      livenessProbe:
        httpGet:
          path: /
          port: 9870
        initialDelaySeconds: 60
        periodSeconds: 10
      readinessProbe:
        httpGet:
          path: /
          port: 9870

```

```

        initialDelaySeconds: 30
        periodSeconds: 5
---
apiVersion: v1
kind: Service
metadata:
  name: namenode
  namespace: bigdata-pipeline
spec:
  type: ClusterIP
  selector:
    app: namenode
  ports:
    - port: 9870
      targetPort: 9870
      name: web
    - port: 9000
      targetPort: 9000
      name: rpc

```

4.2.2 Datanode

```

apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: datanode
  namespace: bigdata-pipeline
spec:
  serviceName: datanode
  replicas: 3
  selector:
    matchLabels:
      app: datanode
  template:
    metadata:
      labels:
        app: datanode
    spec:
      containers:
        - name: datanode
          image: bde2020/hadoop-datanode:2.0.0-hadoop3.2.1-java8
          env:
            - name: SERVICE_PRECONDITION
              value: "namenode:9870"
            - name: CORE_CONF_fs_defaultFS
              value: "hdfs://namenode:9000"
            - name: HDFS_CONF_dfs_permissions_enabled
              value: "false"
            - name:
              HDFS_CONF_dfs_namenode_datanode_registration_ip___hostname___check

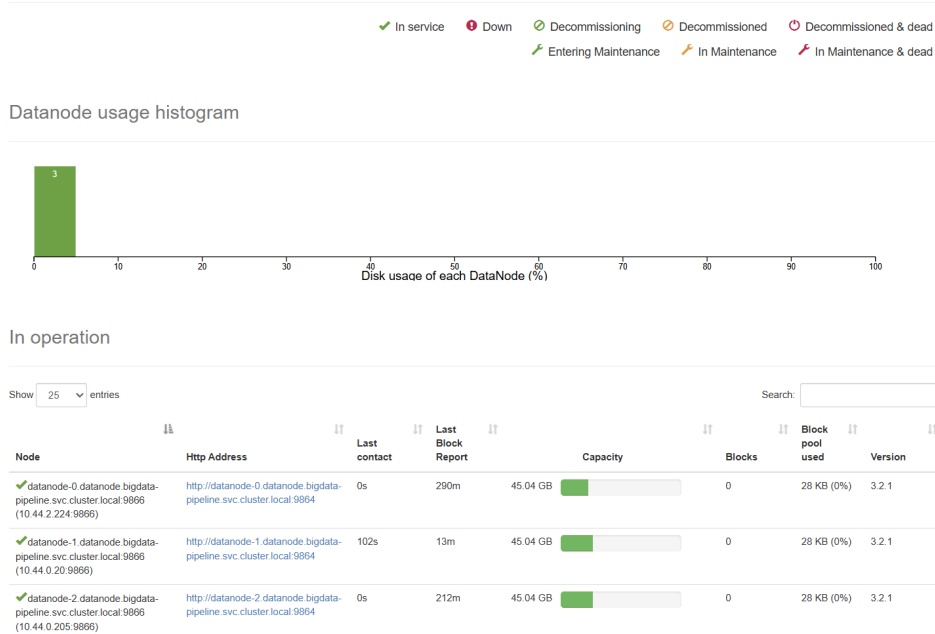
```

```

        value: "false"
resources:
  requests:
    memory: "128Mi"
    cpu: "50m"
  limits:
    memory: "1Gi"
    cpu: "500m"
livenessProbe:
  tcpSocket:
    port: 9864
  initialDelaySeconds: 60
  periodSeconds: 30
readinessProbe:
  tcpSocket:
    port: 9864
  initialDelaySeconds: 30
  periodSeconds: 10
---
apiVersion: v1
kind: Service
metadata:
  name: datanode
  namespace: bigdata-pipeline
spec:
  type: ClusterIP
  selector:
    app: datanode
  ports:
    - port: 9864
      targetPort: 9864
      name: http
    - port: 9866
      targetPort: 9866
      name: ipc
    - port: 9867
      targetPort: 9867
      name: stream

```

Datanode Information



Hình 2: HDFS

4.3 Spark

Our group sets up a Spark cluster consisting of three nodes: one Spark master node and two Spark worker nodes.

4.3.1 Spark Streaming

Spark Streaming is used to generate real-time statistics. Data from the consumer is read into a streaming dataframe and processed, with unnecessary fields being removed.

4.4 Cassandra

Our group sets up a Cassandra cluster consisting of three nodes. One keyspace is created with a replication factor of 3, and within this keyspace, two tables are used to store the information

4.5 Visualization

Since Cassandra is a NoSQL database, and Superset is a visualization tool that relies on SQL queries to manipulate and visualize data, we decide to integrate Trino as an intermediary technology. Trino acts as a powerful distributed SQL query engine that enables seamless translation of SQL queries into operations compatible with NoSQL databases like Cassandra. By leveraging Trino, we can query Cassandra's data efficiently using standard SQL syntax without needing to write complex NoSQL queries manually.

The visualization shows four figures, which are:

- Total Number of Accidents: Displays the overall count of traffic accidents, providing a quick overview of the incident scale.

- **Number of Accidents by Hour:** A bar chart illustrating the distribution of accidents across different hours of the day, showing higher accident frequency during peak hours.
- **Accidents by Weather Condition:** A bar chart categorizing accidents based on weather conditions (Fair, Mostly Cloudy, Rain, and Snow), with Rain contributing the most accidents.
- **Accidents by Severity:** A pie chart highlighting the severity levels of accidents, segmented into categories (e.g., 1, 2, 3).

This dashboard provides actionable insights for understanding trends, peak accident times, and the impact of weather and severity conditions.

5 Conclusion

Through this project, our team has explored, researched, and built a basic data pipeline using technologies such as Kafka, HDFS, Spark, and Cassandra. In the future, we hope to further develop our data pipeline, enabling more extensive data analysis and information aggregation to meet diverse user needs.

6 Problem and Solutions

Problem 1: HDFS Datanode Corruption and Manual Recovery

Solution: The HDFS cluster frequently experienced Datanode corruption, which required manual intervention to restart the affected services, leading to system downtime. Upon analysis, we found that the Datanodes were not configured with self-healing capabilities. To address this, we redeployed the Datanodes using a Kubernetes StatefulSet configuration with a replica count of 3. This setup ensures that if a Datanode pod enters a corrupted or unhealthy state, the Kubernetes scheduler automatically terminates it and provisions a fresh replica to replace it. This automation eliminates the need for manual restarts and maintains the required replication factor for data reliability.

Problem 2: Resource Exhaustion from Individual Record Processing

Solution: The pipeline suffered from frequent crashes and memory exhaustion because the Kafka consumer was writing every incoming record to HDFS individually. This approach generated an excessive number of small files and high I/O overhead, overwhelming the storage system and the consumer pod's computing resources. To resolve this, we implemented a batch processing mechanism based on time windows and message counts. Instead of immediate writes, incoming records are buffered in memory and written to HDFS in consolidated batches. This optimization significantly reduced the frequency of file creation operations, lowered memory pressure, and prevented the pod from crashing due to resource starvation.